

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción: Algoritmos Fundamentales

Recursividad: Fundamento de Algoritmos

Ordenamiento: Bubble Sort

Algoritmos de Búsqueda

Ejercicios Guiados

Actividad Práctica

Consolidación y Retroalimentación

Conclusiones

Introducción: Algoritmos Fundamentales

Introducción: Algoritmos Fundamentales $\ni$ ¿Por qué ordenar y buscar?

Astronomía y Física:

- Ordenar estrellas por magnitud
- Organizar mediciones de laboratorio
- Buscar datos específicos en catálogos
- Analizar distribuciones de datos

Computación:

- Base de algoritmos más complejos
- Optimización de búsquedas
- Análisis de eficiencia
- Fundamento para estructuras de datos

Objetivo de la Sesión

Comprender cómo funcionan los algoritmos básicos de ordenamiento y búsqueda, y analizar su eficiencia.

Introducción: Algoritmos Fundamentales $\ni$ Objetivos de la Sesión

- **Comprender** el concepto de recursividad con ejemplos prácticos
- **Explorar** el algoritmo Bubble Sort paso a paso
- **Implementar** búsqueda lineal y binaria
- **Analizar** la eficiencia de diferentes algoritmos
- **Aplicar** estos conceptos con datos astronómicos

Recursividad: Fundamento de Algoritmos

Recursividad: Fundamento de Algoritmos $\ni$ ¿Qué es la Recursividad?

Definición

Una función es **recursiva** cuando se llama a sí misma para resolver un problema dividiéndolo en subproblemas más pequeños.

Estructura básica de una función recursiva:

1. **Caso base:** Condición que detiene la recursión
2. **Caso recursivo:** Llamada a sí misma con un problema más simple

Importante

Sin un caso base, la recursión sería infinita y causaría un error de desbordamiento de pila.

Recursividad: Fundamento de Algoritmos $\ni$ Ejemplo 1: Factorial Recursivo

Definición matemática del factorial

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

```
1 def factorial(n):
2     # Caso base: factorial de 0 es 1
3     if n == 0:
4         return 1
5     # Caso recursivo: n! = n * (n-1)!
6     else:
7         return n * factorial(n - 1)
8
9 # Ejemplo
10 print(factorial(5)) # 5! = 5*4*3*2*1 = 120
```


Recursividad: Fundamento de Algoritmos $\ni$ Visualización: factorial(5)

Llamadas recursivas paso a paso:

1. **factorial(5)** llama a **factorial(4)**
2. **factorial(4)** llama a **factorial(3)**
3. **factorial(3)** llama a **factorial(2)**
4. **factorial(2)** llama a **factorial(1)**
5. **factorial(1)** llama a **factorial(0)**
6. **factorial(0)** retorna **1** (caso base)

Retornos sucesivos:

1 $\rightarrow$ **$1 \times 1 = 1$** $\rightarrow$ **$2 \times 1 = 2$** $\rightarrow$ **$3 \times 2 = 6$** $\rightarrow$ **$4 \times 6 = 24$** $\rightarrow$ **$5 \times 24 = 120$**

Recursividad: Fundamento de Algoritmos $\ni$ Ejemplo 2: Suma Recursiva de una Lista

```
1 def suma_recursiva(lista):
2     # Caso base: lista vacía suma 0
3     if len(lista) == 0:
4         return 0
5     # Caso recursivo: primer elemento + suma del resto
6     else:
7         return lista[0] + suma_recursiva(lista[1:])
8
9 # Ejemplo con distancias planetarias (UA)
10 distancias = [1.0, 1.52, 5.20] # Tierra, Marte, Júpiter
11 print(suma_recursiva(distancias)) # 7.72 UA
```

Conceptos clave:

- `lista[0]`: primer elemento
- `lista[1:]`: resto de la lista (slicing)
- Cada llamada trabaja con una lista más pequeña

Recursividad: Fundamento de Algoritmos $\ni$ Visualización: suma_recursiva([1.0, 1.52, 5.20])

Descomposición del problema:

1. $\text{suma}([1.0, 1.52, 5.20]) = 1.0 + \text{suma}([1.52, 5.20])$
2. $\text{suma}([1.52, 5.20]) = 1.52 + \text{suma}([5.20])$
3. $\text{suma}([5.20]) = 5.20 + \text{suma}([])$
4. $\text{suma}([]) = 0$ (caso base)

Construcción de la solución:

$$0 \rightarrow 5.20 + 0 = 5.20 \rightarrow 1.52 + 5.20 = 6.72 \rightarrow 1.0 + 6.72 = 7.72$$

Ventaja de la recursividad

Convierte un problema complejo en pasos simples y elegantes.

Ordenamiento: Bubble Sort

Ordenamiento: Bubble Sort $\ni$ ¿Qué es el Ordenamiento?

Definición

Reorganizar una colección de elementos en un orden específico (ascendente o descendente).

¿Por qué es importante?

- Facilita la búsqueda de información
- Permite identificar extremos (máximos/mínimos)
- Base para algoritmos más complejos
- Mejora la visualización de datos

Hoy estudiaremos: Bubble Sort (ordenamiento por burbujeo)

Ordenamiento: Bubble Sort $\ni$ Bubble Sort: Idea Principal

¿Cómo funciona?

Compara **pares de elementos adyacentes** y los intercambia si están en el orden incorrecto. Repite este proceso hasta que la lista esté ordenada.

Analogía visual:

- Imagina burbujas de aire en agua
- Las más ligeras "suben" naturalmente
- Los elementos más grandes "burbujean" hacia el final
- Video demostrativo: <https://www.youtube.com/watch?v=kPRA0W1kECg&t=242s>

Característica clave

Después de cada pasada completa, el elemento más grande está en su posición final.

Ordenamiento: Bubble Sort $\ni$ Bubble Sort: Ejemplo Paso a Paso

Lista inicial:

5 2 8 1 9

Pasada 1:

- Comparar 5 y 2 $\rightarrow$ Intercambiar $\rightarrow$ 2 5 8 1 9
- Comparar 5 y 8 $\rightarrow$ No cambiar $\rightarrow$ 2 5 8 1 9
- Comparar 8 y 1 $\rightarrow$ Intercambiar $\rightarrow$ 2 5 1 8 9
- Comparar 8 y 9 $\rightarrow$ No cambiar $\rightarrow$ 9 está en posición final [?]

Resultado final:

1 2 5 8 9

Ordenamiento: Bubble Sort $\ni$ Implementación de Bubble Sort

```
1 def bubble_sort(lista):
2     n = len(lista)
3     # Realizar n-1 pasadas
4     for i in range(n - 1):
5         # En cada pasada, comparar elementos adyacentes
6         for j in range(n - 1 - i):
7             # Si están en orden incorrecto, intercambiar
8             if lista[j] > lista[j + 1]:
9                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
10    return lista
```

Análisis del código:

- `range(n-1)`: necesitamos n-1 pasadas
- `range(n-1-i)`: cada pasada hace menos comparaciones
- `lista[j], lista[j+1] = ...`: intercambio elegante en Python

Complejidad Temporal

$\mathcal{O}(n^2)$ (complejidad cuadrática)

¿Qué significa esto?

- Para **10 elementos**: ≈ 100 comparaciones
- Para **100 elementos**: $\approx 10,000$ comparaciones
- Para **1,000 elementos**: $\approx 1,000,000$ comparaciones

Ventajas y desventajas

Ventaja: Fácil de entender e implementar

Desventaja: Ineficiente para listas grandes

Ordenamiento: Bubble Sort $\ni$ Ejemplo: Ordenar Distancias Planetarias

```
1 # Distancias en Unidades Astronómicas (UA)
2 dist_au = [1.52, 0.39, 5.20, 1.00, 30.07]
3 planetas = ["Marte", "Mercurio", "Júpiter", "Tierra", "Neptuno"]
4
5 print("Original :", dist_au)
6 print("Planetas :", planetas)
7
8 dist_ordenada = bubble_sort(dist_au.copy())
9 print("\nOrdenado :", dist_ordenada)
10 # [0.39, 1.00, 1.52, 5.20, 30.07]
```

Aplicación:

- Identificar el planeta más cercano/lejano al Sol
- Preparar datos para gráficas
- Facilitar comparaciones entre planetas

Algoritmos de Búsqueda

Algoritmos de Búsqueda $\ni$ Búsqueda: ¿Qué es y por qué importa?

Problema

Encontrar si un elemento específico existe en una colección de datos y determinar su posición.

Dos estrategias fundamentales:

1. **Búsqueda Lineal:** revisar elemento por elemento
2. **Búsqueda Binaria:** aprovechar que la lista está ordenada

Aplicaciones en astronomía:

- Buscar estrella por magnitud en catálogos
- Encontrar exoplaneta con características específicas
- Localizar mediciones en grandes conjuntos de datos

Estrategia

Revisar cada elemento de la lista secuencialmente hasta encontrar el objetivo o llegar al final.

```
1 def linear_search(lista, objetivo):  
2     for i, x in enumerate(lista):  
3         if x == objetivo:  
4             return i # Retorna la posición  
5     return -1 # No encontrado
```

Características:

- Complejidad: $\mathcal{O}(n)$ - lineal
- Ventaja: funciona en listas no ordenadas
- Desventaja: lento para listas grandes

Requisito previo

La lista **DEBE** estar ordenada.

Estrategia "Divide y Conquista":

1. Comparar el objetivo con el elemento del **centro**
2. Si es igual $\rightarrow$ encontrado
3. Si el objetivo es menor $\rightarrow$ buscar en la mitad izquierda
4. Si el objetivo es mayor $\rightarrow$ buscar en la mitad derecha
5. Repetir hasta encontrar o agotar opciones

Ventaja clave

En cada paso, **elimina la mitad** de las opciones restantes.

Algoritmos de Búsqueda $\ni$ Búsqueda Binaria: Ejemplo Visual

Buscar el 7 en:

1 3 5 7 9 11 13

Paso 1:

- Centro: posición 3, valor = 7
- ¡Encontrado! En 1 comparación

Buscar el 11:

Paso 1: Centro = 7 $\rightarrow 11 > 7 \rightarrow$ buscar derecha:

9 11 13

Paso 2: Centro = 11 $\rightarrow$ ¡Encontrado! En 2 comparaciones

Comparación:

- Búsqueda lineal: hasta 7 comparaciones

Algoritmos de Búsqueda $\ni$ Implementación de Búsqueda Binaria

```
1 def binary_search(lista_ordenada, objetivo):
2     low, high = 0, len(lista_ordenada) - 1
3
4     while low <= high:
5         mid = (low + high) // 2 # Índice del centro
6
7         if lista_ordenada[mid] == objetivo:
8             return mid # Encontrado
9         elif objetivo < lista_ordenada[mid]:
10             high = mid - 1 # Buscar en mitad izquierda
11         else:
12             low = mid + 1 # Buscar en mitad derecha
13
14     return -1 # No encontrado
```

Complejidad: $\mathcal{O}(\log n)$ - logarítmica (muy eficiente)

Algoritmos de Búsqueda $\ni$ Comparación de Eficiencia: Lineal vs. Binaria

Tamaño (n)	Lineal $\mathcal{O}(n)$	Binaria $\mathcal{O}(\log n)$
10	10	4
100	100	7
1,000	1,000	10
10,000	10,000	14
100,000	100,000	17
1,000,000	1,000,000	20

Conclusión

Para listas grandes, la búsqueda binaria es **dramáticamente más rápida**.

Ejercicios Guiados

Ejercicios Guiados $\ni$ Ejercicio 1:

Ordenamiento con Datos Simulados



Enunciado

- Generar 5,000 mediciones de temperatura simuladas (distribución normal: media = 22°C, desviación = 2°C)
- Implementar `bubble_sort` y ordenar los datos
- Comparar el tiempo de ejecución con `np.sort()`
- Calcular: temperatura mínima, máxima y mediana

Conceptos: Generación de datos aleatorios, medición de rendimiento, estadística básica.

Aplicación física: Análisis de grandes conjuntos de mediciones experimentales.

Ejercicios Guiados $\ni$ Solución 1 de Referencia:

Preparación de Datos



```
1 import numpy as np
2
3 # Fijar semilla para reproducibilidad
4 np.random.seed(42)
5
6 # Generar 5000 temperaturas con distribución normal
7 temps = np.random.normal(loc=22, scale=2, size=5_000)
8 temps = temps.round(2).tolist() # Redondear y convertir a lista
9
10 print(f"Total de datos: {len(temps)}")
11 print(f"Primeros 5 valores: {temps[:5]}")
```

Discusión:

- `np.random.seed(42)`: garantiza resultados reproducibles
- `.round(2)`: limita a 2 decimales (realista para mediciones)

Ejercicios Guiados $\ni$ Solución 1:

Medición de Rendimiento



```
1 # Tu implementación
2 def bubble_sort(lst):
3     n = len(lst)
4     for i in range(n-1):
5         for j in range(n-1-i):
6             if lst[j] > lst[j+1]:
7                 lst[j], lst[j+1] = \
8                     lst[j+1], lst[j]
9     return lst
```

```
1 # NumPy optimizado
2 import numpy as np
3 arr = np.array(temps)
4 arr_sorted = np.sort(arr)
```

Medir tiempos con %%timeit (Jupyter/Colab)

```
1 %%timeit -n 3 -r 3
2 bubble_sort(temps.copy()) # Tu versión
3
4 %%timeit -n 3 -r 3
5 np.sort(np.array(temps)) # NumPy
```

`%timeit` y `%%timeit` son *magic commands* que extienden las funcionalidades de Jupyter/Colab.

Ejemplos comunes:

- `%timeit`: mide tiempo de **una línea** de código
- `%%timeit`: mide tiempo de **toda una celda**
- `%matplotlib inline`: habilita gráficos en el notebook
- `%ls`, `%cd`: comandos del sistema de archivos
- `%who`: muestra variables definidas

Opciones de `%%timeit`

- n 3: ejecuta 3 veces
 - r 3: repite el proceso 3 veces (total: 9 ejecuciones)
- Retorna: media y desviación estándar del tiempo

Ejercicios Guiados $\ni$ Solución 1:

Cálculo de Estadísticas



```
1 # Ordenar con bubble_sort
2 sorted_temps = bubble_sort(temps.copy())
3
4 # Estadísticas
5 n = len(sorted_temps)
6 temp_min = sorted_temps[0]
7 temp_max = sorted_temps[-1]
8
9 # Mediana (promedio de los dos valores centrales)
10 mediana = (sorted_temps[n//2] + sorted_temps[(n-1)//2]) / 2
11
12 print(f"Temperatura mínima: {temp_min}°C")
13 print(f"Temperatura máxima: {temp_max}°C")
14 print(f"Mediana: {mediana:.2f}°C")
```

Discusión: La mediana es más robusta que la media ante valores extremos (outliers).

Ejercicios Guiados $\ni$ Ejercicio 2:

Búsqueda Binaria en Magnitudes Estelares



Enunciado

- Simular 300 magnitudes estelares (distribución normal: media = 8, desviación = 1.5)
- Ordenar la lista de magnitudes
- Implementar **binary_search** con contador de comparaciones
- Solicitar al usuario una magnitud a buscar
- Comparar número de comparaciones con búsqueda lineal

Conceptos: Búsqueda eficiente, complejidad algorítmica, contador de operaciones.

Aplicación astronómica: Búsqueda de estrellas en catálogos por magnitud aparente.

Ejercicios Guiados $\ni$ Solución 2:

Preparación de Datos



```
1 import numpy as np
2
3 # Generar 300 magnitudes estelares simuladas
4 np.random.seed(42)
5 mags = np.random.normal(8, 1.5, size=300).round(2)
6
7 # Ordenar (requisito para búsqueda binaria)
8 mags_ordenadas = sorted(mags)
9
10 print(f"Total de estrellas: {len(mags_ordenadas)}")
11 print(f"Rango de magnitudes: [{mags_ordenadas[0]}, {mags_ordenadas[-1]}]")
```

Discusión:

- `sorted()`: retorna nueva lista ordenada (no modifica original)
- Magnitud 8: estrellas débiles visibles con telescopios pequeños

Ejercicios Guiados $\ni$ Solución 2: Implementación con Contador



```
1 def binary_search(lst, target):
2     low, high, checks = 0, len(lst) - 1, 0
3
4     while low <= high:
5         mid = (low + high) // 2
6         checks += 1 # Contar cada comparación
7
8         if lst[mid] == target:
9             return mid, checks # Encontrado
10        elif target < lst[mid]:
11            high = mid - 1
12        else:
13            low = mid + 1
14
15    return -1, checks # No encontrado
```

Discusión: El contador `checks` nos permite verificar la eficiencia teórica $\mathcal{O}(\log n)$.

Ejercicios Guiados $\ni$ Solución 2:

Ejecución y Análisis



```
1 # Solicitar magnitud a buscar
2 val = float(input("Ingresa magnitud a buscar: "))
3
4 # Realizar búsqueda binaria
5 pos, nchecks = binary_search(mags_ordenadas, val)
6
7 if pos != -1:
8     print(f"Encontrada en índice {pos}")
9     print(f"Comparaciones realizadas: {nchecks}")
10 else:
11     print(f"No encontrada (se hicieron {nchecks} comparaciones)")
12
13 # Comparar con búsqueda lineal
14 print(f"\nBúsqueda lineal habría necesitado: {pos+1} comparaciones")
15 print(f"Ahorro: {pos+1-nchecks} comparaciones")
```

```
1 def linear_search(lst, target):
2     for checks, x in enumerate(lst, 1):
3         if x == target:
4             return checks
5     return checks # No encontrado
6
7 # Medir tiempos
8 %timeit binary_search(mags_ordenadas, val)
9 %timeit linear_search(mags_ordenadas, val)
```

Resultados esperados (para 300 elementos):

- Búsqueda lineal: hasta 300 comparaciones
- Búsqueda binaria: máximo $\log_2(300) \approx 9$ comparaciones
- **Diferencia:** $\sim 97\%$ menos comparaciones

Actividad Práctica

- Formar **equipos de 2-3 estudiantes**
- Cada estudiante crea su propio notebook
- Tiempo asignado: **25 minutos**
- **Objetivo:** Implementar y comparar algoritmos con datos reales

Metodología

Trabajen juntos en la lógica, pero cada uno escribe su código.
Comparen resultados y discutan diferencias.

Actividad Práctica $\ni$ Actividad Extra 1:

Análisis de Velocidades Radiales



Simular mediciones de velocidades radiales de estrellas para detectar exoplanetas.

Enunciado

- Generar 1000 velocidades radiales (m/s) con distribución normal (media=0, desv=50)
- Ordenar usando `bubble_sort`
- Identificar las 10 mayores variaciones (valores extremos)
- Calcular el tiempo de ejecución
- Comparar con `np.sort()`

Objetivo: Comprender limitaciones de `bubble_sort` con datos reales y ventajas de implementaciones optimizadas.

Actividad Práctica $\ni$ Actividad Extra 2:

Catálogo de Estrellas Brillantes



Trabajar con un catálogo simplificado de estrellas.

Enunciado

- Crear lista de 500 magnitudes aparentes (distribución: media=5, desv=2)
- Ordenar de más brillante a más débil (menor a mayor magnitud)
- Implementar función que busque estrellas en rango específico
- Usar búsqueda binaria para encontrar límites del rango
- Contar cuántas estrellas hay en el rango [4.0, 6.0]

Objetivo: Aplicar búsqueda binaria para consultas de rango, común en análisis astronómico.

Consolidación y Retroalimentación

Consolidación y Retroalimentación $\ni$ Puesta en Común - ¿Cómo les fue?

- ¿Qué algoritmo resultó más difícil de implementar?
- ¿Encontraron errores comunes? ¿Cómo los solucionaron?
- ¿La diferencia de eficiencia fue la esperada?
- ¿Qué ventajas tiene pensar recursivamente?

Objetivo

Aprender de las experiencias colectivas y normalizar los errores como parte del aprendizaje.

En Bubble Sort

- Olvidar `.copy()` al ordenar (modifica lista original)
- Índices incorrectos: `range(n)` en lugar de `range(n-1)`
- No reducir comparaciones: `range(n-1-i)` es crucial para eficiencia

En Búsqueda Binaria

- Olvidar ordenar la lista previamente
- Usar `/` en lugar de `//` para `mid` (debe ser entero)
- Actualizar incorrectamente `low` o `high`
- Bucle infinito si no se actualizan límites

Consejo de depuración

Usar `print()` dentro de los bucles para visualizar el proceso paso a paso.

Conclusiones

- **Aprendimos** el concepto fundamental de recursividad
 - Caso base y caso recursivo
 - Factorial y suma de listas
- **Implementamos** Bubble Sort paso a paso
 - Complejidad $\mathcal{O}(n^2)$
 - Comparación con implementaciones optimizadas
- **Exploramos** búsqueda lineal y binaria
 - Eficiencia: $\mathcal{O}(n)$ vs $\mathcal{O}(\log n)$
 - Importancia del ordenamiento previo
- **Aplicamos** estos conceptos a problemas astronómicos

Habilidad adquirida

Capacidad de analizar y elegir algoritmos apropiados según el tamaño de los datos.

- **Sesión siguiente (P2):** Práctica intensiva con algoritmos más complejos
 - Merge Sort y Quick Sort
 - Análisis comparativo de rendimiento
 - Aplicaciones con datos astronómicos reales
- **Recomendación de estudio:**
 - Visualizaciones: VisuAlgo.net
 - Tutorial: Khan Academy - Algoritmos
- **Práctica sugerida:**
 - Implementar los algoritmos "a mano" con listas pequeñas
 - Experimentar con diferentes tamaños de datos

¿Preguntas o dudas?

Próxima sesión:

Semana 12 - Sesión 2 (P2)

Algoritmos de Ordenamiento Avanzados

¡Nos vemos la próxima clase!